

SPIDR: Five Simple but Powerful Ways to Split User Stories



by Mike Cohn

Tagged:
user stories, news, story splitting, better user stories

One of the most common struggles faced by agile teams is the need to split user stories. I'm sure you've struggled with this. I certainly did at first.

In fact, when I first began using Scrum, some of our product backlog items were so big that we occasionally opted for six-week sprints. With a bit more experience, though, that team and I saw enough ways to split work that we could have done one-day sprints if we'd wanted.

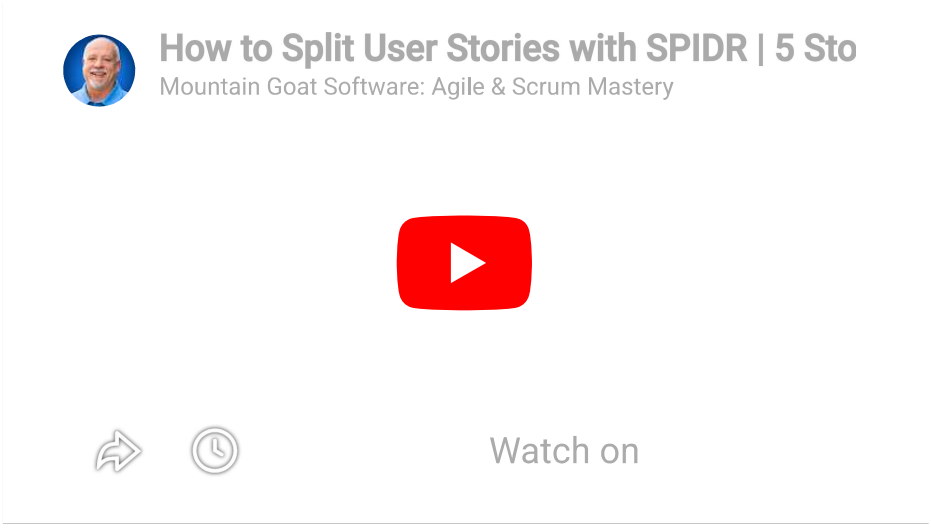
But splitting stories was hard at first. Really hard.

I've got some good news for you. Not only have I figured out how to split stories on my own, I've learned how to explain how to do it so that anyone can quickly become an expert. (Want a peek behind the scenes at real user stories from some of my early product backlogs,

complete with commentary about what I'd do differently today? Download 200+ User Story Examples)

What I discovered is that almost every story can be split with one of five techniques. Learn those five simple techniques and you're set.

Even better, the five techniques form an easily memorable acronym: SPIDR. I introduce each technique below, and the video shows them in action.



SPIDR Technique for Splitting Stories

A few years ago I was creating the Better User Stories course. Because this course would cover everything someone needs to know to work effectively with stories, I knew it needed a module on splitting.

To create that module, I printed out over a thousand user stories I'd collected over 15 years. For each story, I had the original story and the sub-stories it had been split into. I taped each story onto the wall, grouping them based on how they'd been split. I was looking for the common approaches used in splitting all these stories. I went through a variety of groupings, trying to find the smallest set of approaches possible. I knew it would be easier to remember 5 splitting techniques rather than 20.

The five I ended up with form the acronym SPIDR—S, P, I, D and R—spider without an E. Let's take a look at the five splitting user stories techniques that make up the SPIDR acronym, with examples of how your team might use them.

1. Splitting User Stories Using a **S**pike

S is for Spike. That's one most agile teams are familiar with. A spike is a research activity a team undertakes to learn more about some backlog item. Spikes can also give teams the knowledge they need to split a complex story. Think of it as a research activity, but it may include prototyping or some experimental coding. During a spike a team isn't trying to develop the new functionality, instead they are developing new knowledge that will help them develop the functionality later.

Take YouTube for example. Go back in time to when YouTube added automatic captioning. The team doing that might have faced a build vs. buy decision. Do they use some commercially available software to generate the captions? Or are their needs so unique that they need to develop something from scratch? The way to settle that would be a spike to test out one or more commercially available captioning products.

Extracting a spike makes the original story smaller because some or all of the research included in the original story is removed. This is absolutely an essential way to split stories. So extracting a spike is one of the five splitting techniques you should use. But normally it won't be the first technique you'll reach for.

2. Splitting User Stories by **P**ath

P is for Path. If a user can do something in multiple ways (for example, paying with a card vs Apple Pay), that's a great area for splitting.

To split a story by paths, look for alternate paths through the story. Sticking with YouTube, let's use the story, "I can share a video with my friends."

When I click the "Share" button in YouTube today, I'm shown 14 buttons I can click to share directly to various social networks. I'm also shown a link I can copy. And I'm given the option to customize that link to start playback of the shared video at a specific time within the video.

That's 16 different paths through the "I can share a video" story. I don't know that this story needs to be split into that many smaller sub-stories. That's for the team to decide based on the effort involved. But,

with the path technique alone we've identified 16 paths through the original story.

3. Splitting User Stories by **I**nterfaces

I is for Interfaces: Splitting your story by browser, or hardware, or delivering a complex interface in iterations. An example might be delivering a version that only works in Chrome this iteration, and saving Safari for another iteration.

In other cases, splitting by interface means creating a simple version of the interface and a more involved version as separate stories. This usually applies to the user interface.

Applying this to our YouTube video sharing example, as an alternative to splitting that story by paths, we could have split out a basic sharing story like, "As a video viewer, I can get a URL I can share." This could be implemented with no user interface other than a share button on the video page. The popup with the 16 different ways of sharing wouldn't be needed if the only way to share is with a URL.

A subsequent story could be, "As a viewer, I can share a video to various social media sites." This could be done with a very simple user interface at first—no fancy scrolling through a list of logos, maybe just a dropdown list of text with the names of the social sites.

The final story could then be something like, "As a viewer, I can choose the social network to share to by scrolling through a list showing the logos of each."

Splitting by interface works because the ultimately desired feature can be built up to by progressively more detailed, and better, interfaces.

4. Splitting User Stories by **D**ata

The D of the SPIDR acronym is for Data. To split a story by data, consider whether you can deliver value in an iteration by simplifying or restricting the data that's supported. Perhaps you can do an initial version of a story that processes only a subset of the data that will ultimately need to be supported. For example, don't allow negative bank balances in the first iteration. Add support for those with a different user story in the next iteration.

Returning to the YouTube example, YouTube allows you to upload a video in any of 16 different file formats. If we're building a YouTube competitor, screw 16 file formats. Let's start with 1. We're going to support one type of data. All uploads need to be in MP4 format for now. We'll add all the others later as separate stories.

Splitting by data is an effective approach. Often there are a few types of data that add a lot of complexity. Well, do an initial implementation that ignores the more complex data. Get that working then add support for the more complex data. You probably can't release the simpler version but you can still build it in that order.

I worked on a human resources system that did exactly this. The system tracked who the manager was for each employee and would do things like route time off requests to that manager. Most employees have one manager but some employees had multiple managers. We needed to support having multiple managers but some stories were simplified initially by assuming each employee had exactly one manager.

5. Splitting User Stories by **R**ules

R is for Rules. Temporarily relaxing support for the rules that a story will ultimately need to support can make large stories smaller.

Sticking with YouTube as an example, YouTube has some strict rules around including copyrighted music in videos. If we are building a competitor to YouTube, our team's first story will be, "I can upload a video so that others can watch it." That story probably sounds simple but there's a lot to it.

So in the first iteration, let's ignore the rule that videos can't contain copyrighted music. We're not announcing our new YouTube competitor to the world after only one iteration anyway. We'll have plenty of time after this first sprint to comply with our internal rule about not allowing videos with copyrighted music.

As another YouTube-related example, suppose we want to prevent certain text from appearing in comments. That could be swearing or maybe SQL commands that could be a hacking attempt. Great idea: let's protect our users and our system from this type of text in comments. But an initial story of "As a user I can enter a comment on a video" can ignore that rule. Doing so makes the story smaller so that

it can fit within an iteration. And support for the rule can be added a couple of iterations later.

Try Story Critic on a Backlog Item

Paste in a user story, job story, or technical story (with or without acceptance criteria). The Story Critic gives you fast feedback on clarity, readiness, and backlog position.

Submit your user story, i.e. As a user... Go

Read more about the Story Critic and download a skill you can add to ChatGPT or Claude. Create a free MGS Essentials account for unlimited access to the Story Critic and other tools.

Getting Better at Splitting Stories

Getting better at splitting user and job stories is an important skill. With the short iterations used in agile, it's helpful to have small units of work. The five techniques we've covered here—splitting by spikes, paths, interfaces, data, and rules—should allow you to split any story.

The SPIDR techniques are easy to remember but putting them into action can require a little training and a lot of practice. That's why I put together a Better User Stories video course that includes the SPIDR method for splitting stories, and a whole lot more.

Last update: April 22nd, 2026



About the Author

Mike Cohn specializes in helping companies adopt and improve their use of agile processes and techniques to build extremely high-performance teams. He is the author of User Stories Applied for Agile Software Development, Agile Estimating and Planning, and Succeeding with Agile as well as the Better User Stories video course. Mike is a founding member of the Agile Alliance and Scrum Alliance and can be reached at hello@mountaingoatsoftware.com. If you want to succeed with agile, you can also have Mike email you a short tip each week.